Individual Project
Data Integration B142

Olist E-Commerce Data Integration: ETL Pipeline Design and Business Insights Using Apache Spark

GitHub repository: https://github.com/danielarubio2005/olist-data-integration
Video: https://youtu.be/CoYDLGlEI_s

Daniela Rubio Sánchez
GH1024883

Table of Contents

Introduction

Olist is a leading Brazilian E-commerce enabler, founded in 2015 (Olist, 2026), that "connects small and medium-sized businesses to customers" (MetaIt, 2024), and major marketplaces such as Mercado Livre, Amazon, Americanas, and Submarino (Remote in Tech, 2022). Like other e-commerce platforms, Olist generates massive volumes of operational data spread across many systems, including orders, payments, reviews, inventory, logistics, among others. Therefore, without integration of all these data, it would be impossible to get insights, analyze it coherently and make it useful to guide business decisions and strategy.

Accordingly, in order to process and work with this large amount of data, big data tools need to be implemented; indeed, traditional tools, like Python pandas, hit memory limits past a few million rows, while tools like Apache Spark provide distributed, in-memory processing that scales linearly, enabling big data workloads. Hence, to handle the data generated by Olist, such big data tools are fundamental.

Consequently, the objective of this project is to build an end-to-end ETL pipeline using PySpark on Databricks, that ingests the Olist Brazilian E-Commerce dataset (consisting of approximately 100k orders across 9 source tables) published in Kaggle[1] (2021), integrates it into a single analytical fact table, and produces actionable business insights. In this project, the aim is to answer the 6 following questions:

1. How are sales evolving over time? (Trend analysis)
2. Which product categories generate the most revenue? (Top N analysis)
3. Which Brazilian states are the biggest markets? (Geographic distribution)
4. How does delivery time relate to customer satisfaction? (Correlation)
5. What payment methods do customers prefer? (Distribution)
6. Are sellers delivering on time? (Operational performance)

These all map to relevant business decisions that Olist should consider for the business strategies. Furthermore, the scope of the project will be limited to delivered orders to make meaningful analysis considering delivery time and disregard canceled orders for other metrics.

System Design

The chosen public dataset from Olist Brazilian E-Commerce, downloaded from Kaggle, consists of about 100k orders, during the period 2016-2018, and 9 CSV files covering orders, customers, sellers, products, order items, payments, reviews, geolocation, and a Portuguese-to-English category translation, forming approximately 1.5M rows in total, where geolocation dominates.

[1] https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce

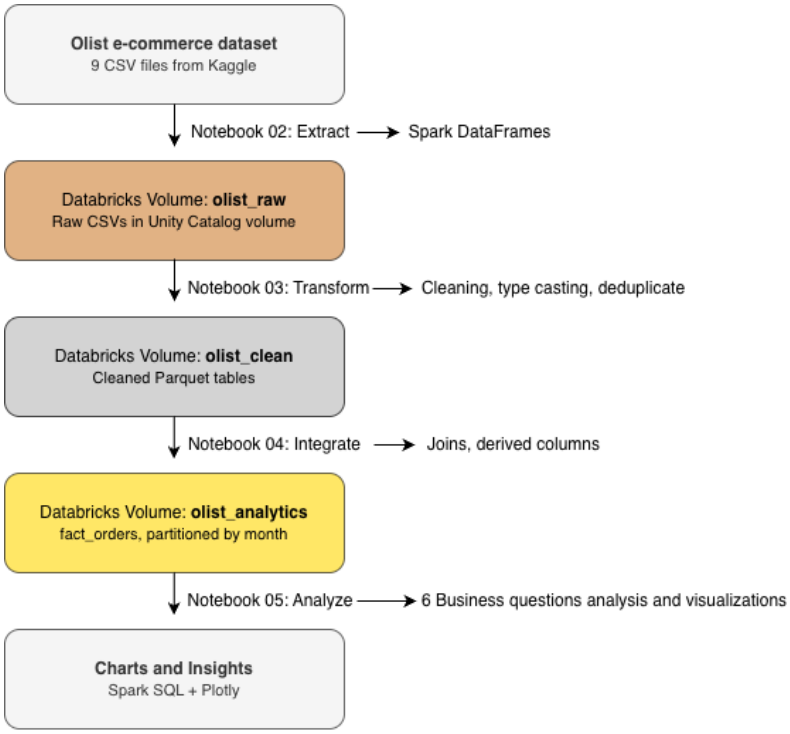


Figure 1: Own illustration made with draw.io /Access here https://drive.google.com/file/d/1V4i2IsIFBRUeT3i8dpd6ckM_pNkfkuGm/view?usp=drive_link

Figure 1 illustrates the end-to-end architecture of the data integration pipeline created for this project. The design follows the "medallion architecture pattern", a widely adopted Databricks reference design that organizes data into three progressive quality levels: bronze (raw), silver (cleansed), and gold (business-ready) (Databricks, 2022). Each level is materialized as a separate Unity Catalog Volume in the Databricks workspace, and each transition between levels is executed by a dedicated PySpark notebook. Indeed, separating the process in this way made it easier to ensure that every stage of the pipeline is independently testable, reproducible, and easy to reason about.

The source layer consists of the nine CSV files named above. Then, Notebook 02 (Extract) ingests these CSVs into Spark DataFrames using schema inference and stores them unchanged in the "bronze tier" (olist_raw). This is important since keeping raw data immutable preserves a reliable history and allows the pipeline to be re-run from any later stage without re-downloading source files.

Afterwards, Notebook 03 (Transform) clean and fixes the data to produce the "silver tier" (olist_clean): timestamp parsing, deduplication, schema alignment between the products table and the Portuguese English category translation, retention of the latest review per order, and standardization of geolocation records to one row per zip-code prefix. Then, the cleaned tables are saved as Parquet files, a columnar storage format that offers significantly faster reads and uses less space than CSVs (Holanda, 2024).

Next, Notebook 04 (Integrate) materializes the "gold tier" (olist_analytics) by aggregating order-line and payment data into a single row for each order, joining the cleaned silver tables on shared IDs (natural keys), and adding derived columns such as delivery_days, delivery_delay_days, and date parts. The resulting fact_orders table is partitioned by purchase_year and purchase_month to make future searches much faster as the system can skip over data it doesn't need, and time-based queries are most common.

Last, Notebook 05 (Analyze) reads the "gold table" by registering it as a Spark SQL temporary view and executing six analytical queries that answer the different business questions. Ultimately, results are turned into interactive Plotly visualizations and exported for inclusion in this report.

Moreover, the technology stack was chosen for efficiency and ease of use. Databricks Free Edition provides a serverless Spark environment with no infrastructure setup and supports Unity Catalog Volumes. Also, PySpark and Spark SQL were used for their optimized, declarative processing via the Catalyst engine (Shaik Sameer, 2025), while, as explained, Parquet was selected as the storage format for its columnar format, its reduced space and because it supports partition pruning for faster queries. Finally, Plotly enables the creation of interactive HTML charts that can be easily exported to static images for the report.

Implementation

The implementation of the ETL pipeline was done in 4 main Notebooks plus one for the dataset exploration in DataBricks; "01_explore_raw_data", "02_extract", "03_transform", "04_load_integrate" and "05_analytics_and_visualization".

The 01_explore_raw_data notebook performs initial data exploration to verify the raw data files are accessible and readable, as a quick validation step before building the full ETL pipeline. For this, there is just one cell that read the orders dataset from the olist_raw Volume, counts rows, displays the first five records, and prints the inferred schema. Therefore, it is ensured that the Volume path is correct and accessible, the CSV are formatted properly, Spark can read and infer schemas automatically and the data look reasonable before proceeding with the full extraction pipeline.

Then, the 02_extract notebook performs the "Extract" step of the pipeline for the Olist data. Here, it loads the 9 CSV files from Databricks Volumes and performs initial data profiling; first, the set up and path definitions is done, creating a dictionary with all the files' paths to then loop through the dictionary and load each CSV using "spark.read.csv" with "header=True" and "inferSchema=True", storing DataFrames in "dfs" dictionary and printing rows and column counts for each dataset. It is worth noting that Spark applies lazy evaluation, so the "spark.read.csv" calls do not actually read the files at the point of invocation; they only build a logical plan, and the data is materialized only when an action such as count() or show() is triggered, which is why the row counts in the next step are the first operations to incur real Input/Output. Afterwards, the first 3 rows of each schema are displayed as a sample data for visual inspection, with column names and inferred types, showing already some data quality issues, such as order_purchase_timestamp being a string. Then, null counts on all 9 datasets is performed, to continue initial data quality assessment, revealing some more key findings like reviews having 88% missing comment titles and 60% missing comment messages, suggesting that people just leave a star rating, also, approximately 2% of orders missing delivery dates information, so orders that weren't delivered yet, and ~2% of products missing category names and dimensions, while customers, geolocation and sellers have no nulls (as shown in figure 2); all these will be handled in the next Transform step in the next notebook.

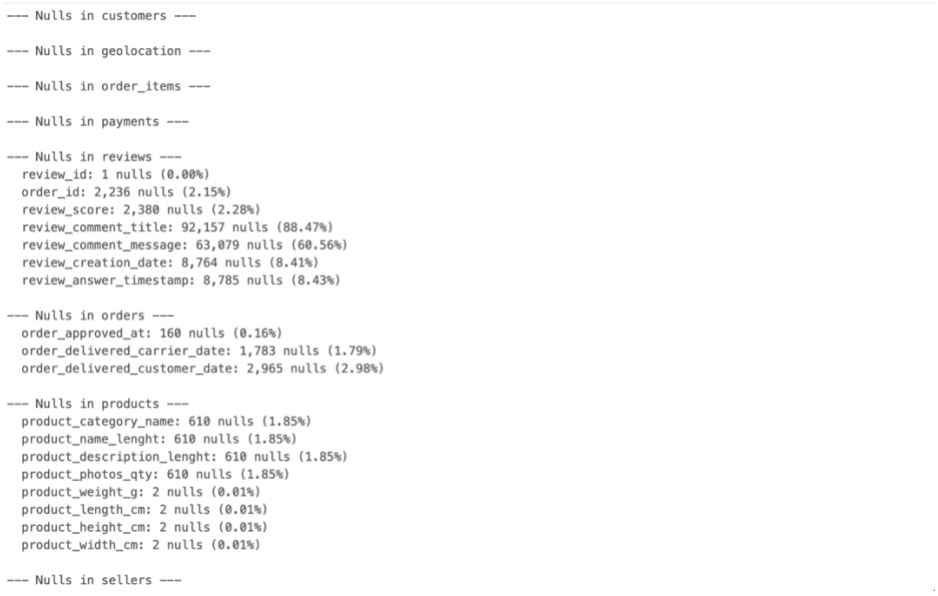


Figure 2: Screenshot of cell 4 of Notebook 02 "02_extract" of DataBricks project on Olist data.

Thereafter, the 03_transform notebook performs the transform and clean step of the ETL pipeline, by loading the raw CSV files and applying data quality transformations, to output cleaned Parquet files. In the first cell, when loading the data, it was noted that the reviews file required multiLine=True, escape= ' " ' due to embedded newlines in Portuguese customer comments (challenge discussed in the next section), so multiLine CSV parsing for text fields was used. Then, the cleaning operations done in this notebook are converting timestamp strings to native TimestampType and deduplicating records using dropDuplicates in all tables that required it, filtering orders to delivered status (focusing analytics on completed transactions) and, for Reviews, using a window function (partitioned by order_id, ordered by review_answer_timestamp descending) to keep only the latest review. Furthermore, to convert Portuguese category names to English, a left join between Products and Category Translation table was performed, and the missing categories were filled with "unknown" to not loose any product that doesn't have a translation, and missing dimensions with 0. Also, to clean geolocation, cell 7 aggregates multiple lat/lng entries per zip code by averaging coordinates, reducing to 19,015 unique zip prefixes. Lastly, the 8 cleaned DataFrames (not 9 as there was a join with the translation) are saved to Parquet format at /Volumes/workspace/default/olist_clean for downstream reuse, and this becomes the input for the next notebook and step of the pipeline.

Subsequently, the 04_load_integrate Notebook performs the load and integrate step of the ETL pipeline, so this is where a denormalized analytics fact table is created by joining the multiple cleaned datasets. First, the cleaned Parquet tables are loaded to then aggregate payments per order (handling multiple payments per order into single row grouped by order_id), and aggregate order items per order (also into single rows grouped by order_id). Then, the orders table is joined with customers, aggregated items, aggregated payments, reviews, products, and sellers using left joins on natural keys, all into a single fact_orders table that contains customer information, order details, payment totals, review scores, product categories and seller locations. It is important to note the usage of left join, as it is important to keep all orders even if reviews or payments are missing. Thereafter, useful derived metrics are added, including delivery_days, delivery_delay_days, purchase_year/month/dayofweek, as these can enrich the analysis related to delivery time. Lastly,

the final table is written to Parquet partitioned by year/month, considering that time-based queries are more common, so implementing partitioning by this way will make future queries that filter by date faster. To verify, a last cell is added to validate the written data, confirming total row count, column count and data range covered.

Finally, the last notebook 05_analytics_and_visualization, registers the fact table as a Spark SQL temporary view, installs Plotly to create the charts to visualize the data for analysis (the DataBricks free edition has matplotlib by default but Plotly was chosen for visually more attractive graphs), and runs 6 analytical SQL queries to answer the business questions of this project. Hence, the queries cover revenue trends, top categories, geographic distribution, the delivery–satisfaction relationship, payment methods, and on-time performance. Accordingly, each result is visualized as one chart, relevant to the question and type of data, with Plotly. The detailed findings from each of these six queries are presented in Section 5: Results.

Challenges and Solutions

During the development of the pipeline, several challenges were encountered, each of which required a specific design decision to resolve.

For instance, there were multi-line text in reviews CSV, so malformed CSV parsing. The most disruptive issue arose when transforming the "olist_order_reviews" file, because customer review comments are free text written in Portuguese, and a number of these comments contained embedded commas, quotation marks, and newline characters. Since Spark's default CSV reader interpreted these embedded newlines as record terminators, rows split prematurely and shift comment text into adjacent columns. This misalignment produced a "CAST_INVALID_INPUT" error (see Figure 3) when the misplaced text was cast to "TimestampType". The issue was resolved by re-reading the file with the "multiLine=True" and "escape='"'" options (as shown in Figure 4), instructing Spark to respect "RFC 4180 quoting rules" (Shafranovich, 2005) and treat quoted fields as atomic, regardless of internal punctuation of the reviews.

Figure 3: "CAST_INVALID_INPUT" error when cleaning Order Reviews dataset. Screenshot of Cell 4 of Notebook 03 "03_Transform" of DataBricks project on Olist data.

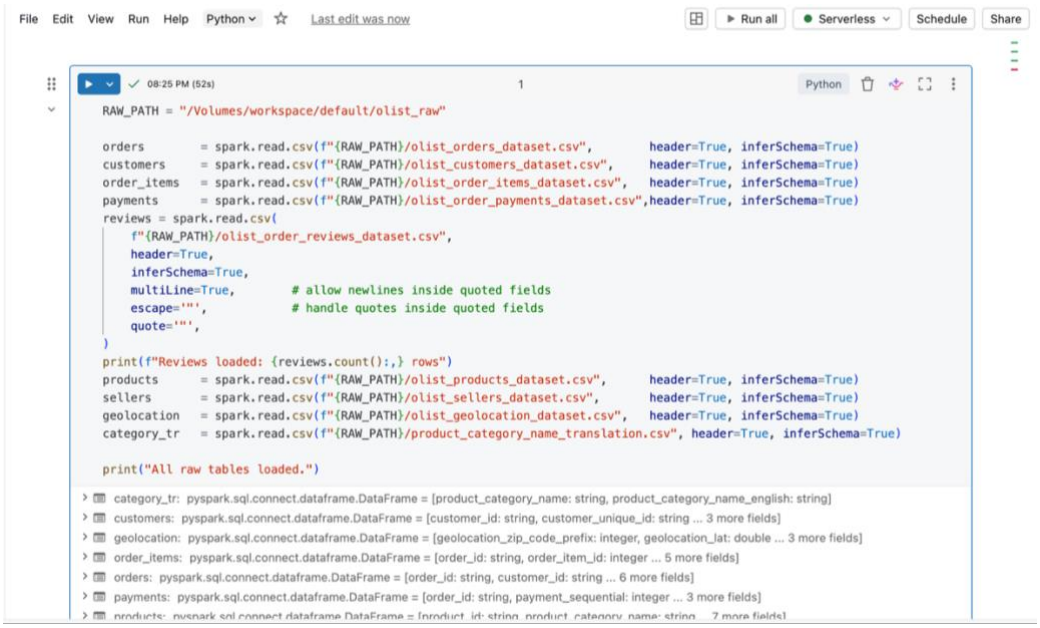
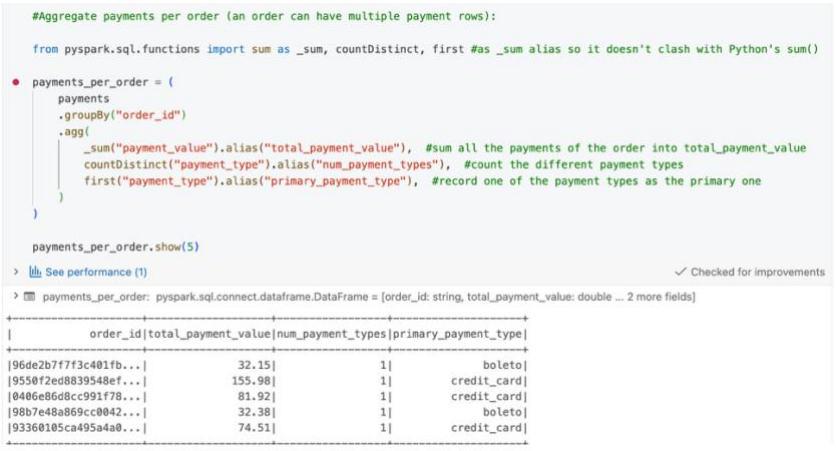


Figure 4: Re-read the Orders Reviews file to honor RFC 4180 quoting rules. Screenshot of Cell 1 of Notebook 03 "03_Transform" of DataBricks project on Olist data.

Additionally, there was schema misalignment between product categories; the products table stores category names in Portuguese, which limited the interpretability of any analysis using categories. However, a separate translation file mapped these to English equivalents, so this was resolved by performing a left join between the two tables on product_category_name and applying "coalesce" to substitute the value "unknown" wherever a translation was absent (as shown in Figure 5), ensuring no category records were lost.

Figure 5: Left join between Products and Category Translations on product_category_name. Screenshot of Cell 5 of Notebook 03 "03_Transform" of DataBricks project on Olist data.

Also, a single order can contain multiple items and multiple payment entries, meaning that the orders, order_items, and order_payments tables exist at different levels of granularity. Therefore, joining them directly would have duplicated order records, thus the solution was to first aggregate order_items and order_payments to one row per order_id, (summing prices, freight, and payment values) before joining them to the orders table (see Figure 6 and 7), thereby preserving a consistent one-row-per-order grain in the final fact table.



```
#Aggregate payments per order (an order can have multiple payment rows):

from pyspark.sql.functions import sum as _sum, countDistinct, first #as _sum alias so it doesn't clash with Python's sum()

payments_per_order = (
    payments
    .groupBy("order_id")
    .agg(
        _sum("payment_value").alias("total_payment_value"),  #sum all the payments of the order into total_payment_value
        countDistinct("payment_type").alias("num_payment_types"),  #count the different payment types
        first("payment_type").alias("primary_payment_type"),  #record one of the payment types as the primary one
    )
)

payments_per_order.show(5)
```

See performance (1) ✓ Checked for improvements

payments_per_order: pyspark.sql.connect.dataframe.DataFrame = [order_id: string, total_payment_value: double ... 2 more fields]

order_id	total_payment_value	num_payment_types	primary_payment_type
96de2b7f7f3c401fb...	32.15	1	boleto
9550f2ed8839548ef...	155.98	1	credit_card
0406e86d8cc991f78...	81.92	1	credit_card
98b7e48a869cc0042...	32.38	1	boleto
93360105ca495a4a0...	74.51	1	credit_card

Figure 6: Aggregate payments per order. Screenshot of Cell 2 of Notebook 04 "04_load_integrate" of DataBricks project on Olist data.

```
#Aggregate order items per order (an order can have multiple items):

from pyspark.sql.functions import count

items_per_order = (
    order_items
    .groupBy("order_id")
    .agg(
        count("order_item_id").alias("num_items"),
        _sum("price").alias("total_price"),
        _sum("freight_value").alias("total_freight"),
        first("seller_id").alias("primary_seller_id"), #for orders with multiple items, I picked one seller/product as
        first("product_id").alias("primary_product_id"), #representative to keep the fact table at one row per order
    )
)

items_per_order.show(5)
```

See performance (1) ✓ Checked for improvements

items_per_order: pyspark.sql.connect.dataframe.DataFrame = [order_id: string, num_items: long ... 4 more fields]

order_id	num_items	total_price	total_freight	primary_seller_id	primary_product_id
0084e195fbd72ae51...	1	89.9	23.13	bd0389da23d89b726...	a963ea4d494221de0...
01a0013ddc7cd129e...	3	1737.0	73.5	a4b6b9b992b46e9ef...	152397b614be35e54...
02c8a77069dc7fd1c...	1	279.99	15.55	46dc3b2cc0980fb8e...	17823ffd2de8234f0...
03ce210a1614bb5a5...	1	499.0	59.16	5dceca129747e92ff...	c874b6cca27efc978...
0406e86d8cc991f78...	1	69.0	12.92	058fd0aa2bfdb2274...	d6c2073fdc29a2931...

Figure 7: Aggregate order items per order. Screenshot of Cell 3 of Notebook 04 "04_load_inetgrate" of DataBricks project on Olist data.

Besides, there were some duplicate review records because some orders were associated with more than one review entry, as customers are occasionally able to update a previously submitted review, but retaining all entries would have skewed average review scores. Hence, a window function partitioned by order_id and ordered by review_answer_timestamp in descending order was applied, using row_number() to retain only the most recent review per order, as shown in Figure 8. Is important to note that, not using a window function will imply using groupBy

("order_id").agg(max("review_answer_timestamo")) to find the latest timestamp per order, and then join back to the original reviews table to get the full row, while the window sort reviews for each order_id by answer date (newest first), number them 1, 2, 3…., and keep only the rows numbered 1.

```
#Clean reviews: data types and reviews can have duplicates per order_id from updates, so I'll keep latest

from pyspark.sql.window import Window
from pyspark.sql.functions import row_number

reviews_clean = (
    reviews
    .withColumn("review_creation_date",   to_timestamp("review_creation_date"))
    .withColumn("review_answer_timestamp", to_timestamp("review_answer_timestamp"))
)

# Keep only the latest review per order (.partitionBy("order_id") ensures to keep only the latest review for each order,
not across all reviews, and descending so it's the last)
w = Window.partitionBy("order_id").orderBy(col("review_answer_timestamp").desc_nulls_last())
reviews_clean = (
    reviews_clean
    .withColumn("rn", row_number().over(w))
    .filter(col("rn") == 1)
    .drop("rn")
)

print(f"Reviews after droping duplicates: {reviews_clean.count():,}")
```

```
> See performance (1)                                                    Optimize
> reviews_clean:  pyspark.sql.connect.dataframe.DataFrame = [review_id: string, order_id: string ... 5 more fields]
Reviews after droping duplicates: 98,673
```

Figure 8: Application of a window function partitioned by order_id and ordered by review_answer_timestamp in descending order to keep the latest review per order. Screenshot of Cell 4 of Notebook 03 "03_transform" of DataBricks project on Olist data.

Finally, in terms of platform and storage configuration, Databricks Free Edition manages data through Unity Catalog Volume rather than the legacy DBFS file paths, so this required each volume (olist_raw, olist_clean and olist_analytics as seen in Figure 9) to be created explicitly before any read or write operation. Once the correct Volume-based path convention was adopted, all storage operations executed reliably.

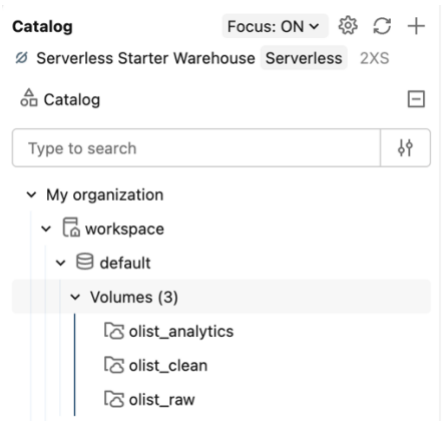


Figure 9: Screenshot of Volumes of DataBricks project on Olist data.

Overall, the challenges faced were accurately tackled to get the best outcome for the analysis of the data and the results, still considering that the Olist raw dataset was in general pretty

complete and manageable, so there were less challenges faced as it would be if working with a dataset that must be further cleaned or preprocessed.

Results

After all the data was transformed, integrated and loaded, the analysis of the six analytical SQL queries was executed against the integrated fact_orders table. Each query addresses a distinct business question, and the corresponding visualization is discussed below.

For the first question, "How sales are evolving over time?", the monthly revenue analysis shows that Olist experienced strong sustained growth through 2017 and into 2018, with a pronounced spike of 1.153393M BRL in November 2017, consistent with Black Friday seasonal demand. This indicates a healthy growth trajectory and confirms that seasonal events represent a significant share of annual revenue, which should be considered for inventory and capacity planning.

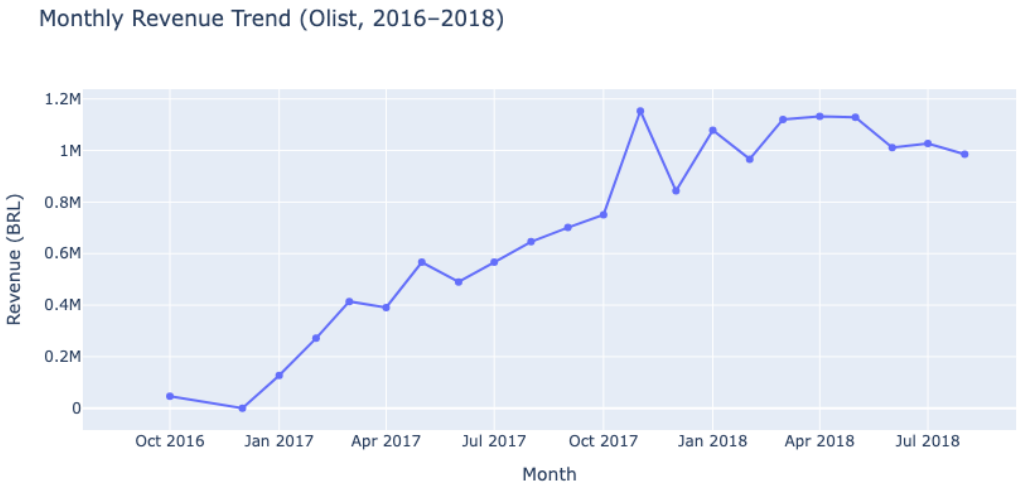


Figure 10: Monthly revenue trend 2016-2018. Own graph created with Plotly.

Then, to discuss question two, "Which product categories generate the most revenue?", the revenue was aggregated by product category, revealing that the platform's income is concentrated in a small number of categories, with health and beauty being number 1, then watches/gifts and, in third place, bed/bath/table, generating the highest revenue. However, the average review score varies across the categories (right colored bar), indicating that high revenue does not always coincide with high customer satisfaction. This identifies specific categories, like beauty and health, where seller-quality interventions could protect revenue.

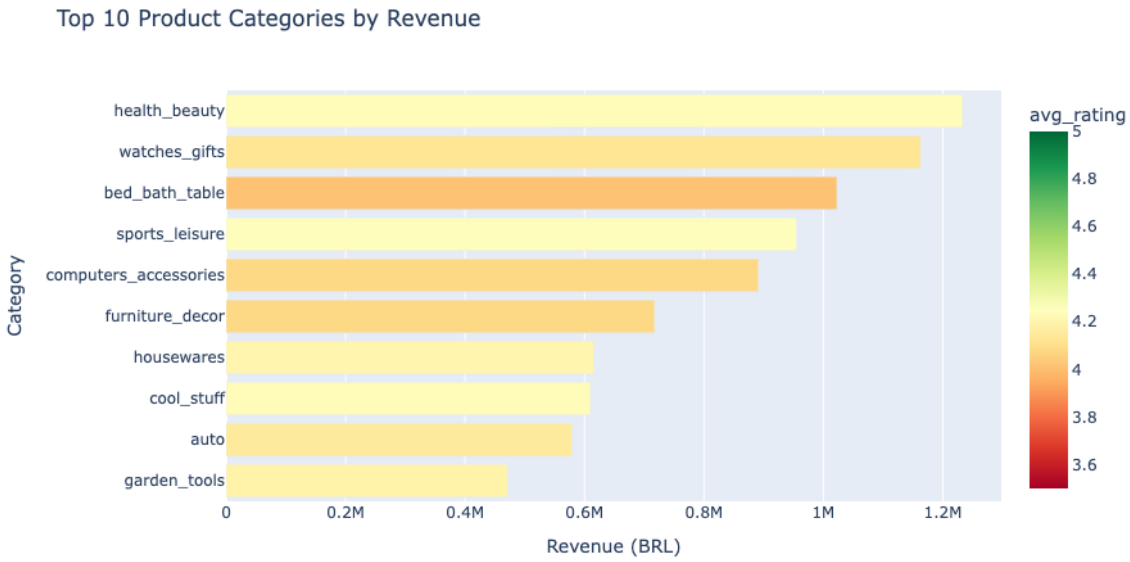


Figure 11: Top product categories (by revenue) and corresponding average ratings. Own graph created with Plotly.

In terms of geographic distribution, to know which Brazilian states are the biggest markets, the analysis of revenue by customer state shows a strong geographic concentration: the state of São Paulo (SP) alone accounts for approximately 37.41% of total revenue, followed by Rio de Janeiro (RJ) and Minas Gerais (MG). While this reflects Brazil's population distribution (Worldpopulationreview.com, 2024), it also represents a concentration risk and highlights underserved states as potential expansion opportunities.

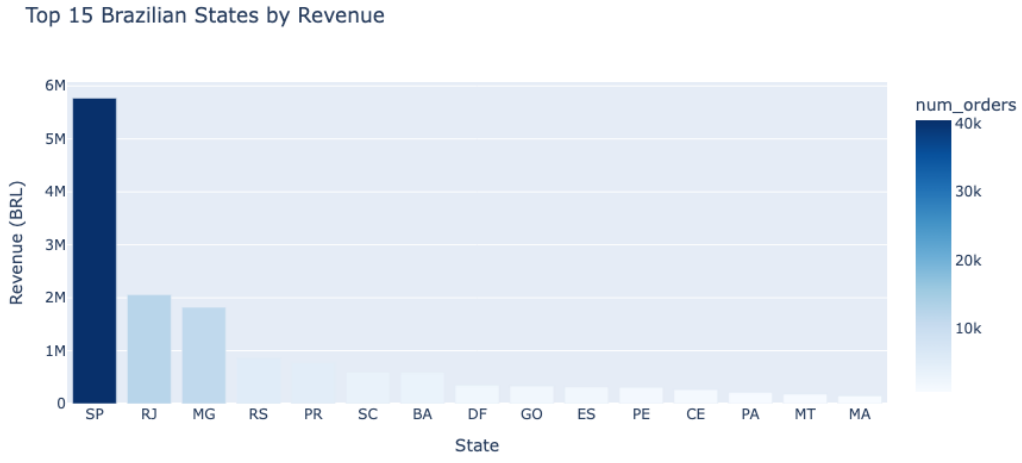


Figure 12: Top 15 Brazilian States by revenue. Own graph created with Plotly.

Moreover, the analysis of the relation between delivery time and customer satisfaction shows a clear inverse relationship between delivery time and review score: orders rated five starts were delivered in an average of 10.6 days, whereas orders rated one star averaged 20.1 days, over 1.9 times longer. This identifies delivery speed as the single strongest operational lever for improving customer satisfaction.

Figure 13: Average delivery time vs. customer review score. Own graph created with Plotly.

Additionally, the distribution of payment methods show that credit card is the dominant method, used in 75.9% of orders, followed by Boleto (a Brazilian bank-slip payment (Stripe.com, 2024)) at 19.9%, whereas voucher and debit card payments are comparatively rare. Hence, this informs prioritization decisions for payment-system reliability and partnership investments.

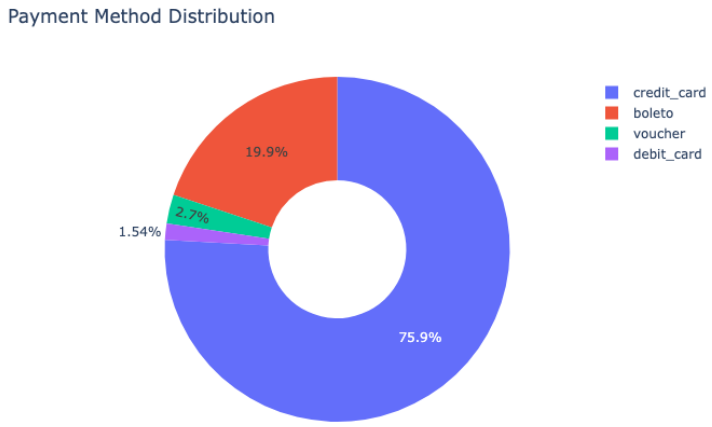


Figure 14: Payment method distribution. Own graph created with Plotly.

Lastly, the on-time delivery rate remained consistently high, above 90% for most of the observed period. However, a noticeable decline is visible in late 2017, coinciding with the Black Friday revenue peak identified in Figure 10. This demonstrates a capacity constraint during periods of peak demand and reinforces the need for seasonal logistics planning.

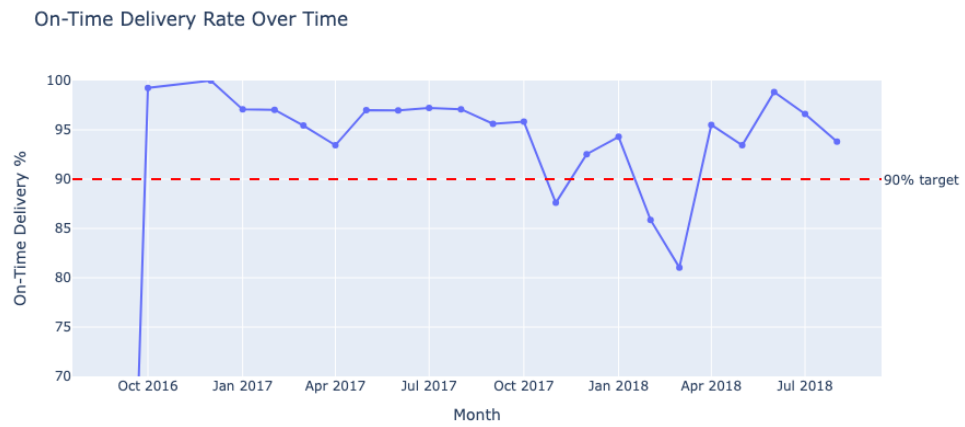


Figure 15: On-time delivery rate over time (2016-2018). Own graph created with Plotly.

Overall, each analytical query was accurate to answer the corresponding business question and, based on the visualization of the analysis, the insights for the business and what each means from Olist perspective was evaluated.

Furthermore, the complete pipeline (ingestion, transformation, integration, and analytics) executed end-to-end in approximately 2 minutes 19 seconds on Databricks Free Edition serverless compute. Notably, the use of Parquet with year/month partitioning evidently improved query response times in the analytics stage compared with reading the original CSV files, confirming the benefit of the storage decisions described in the System Design section.

Conclusion and Future Work

This project successfully designed and implemented an end-to-end big data integration pipeline using Apache Spark on Databricks. The pipeline ingested the Olist Brazilian E-Commerce dataset (nine separate source files comprising approximately 100,000 orders) and integrated them into a single, fact table ready for analytics through a sequence of extraction, cleaning, transformation, and integration stages. By following the “medallion architecture pattern”, the design maintained a clear separation between raw, cleansed, and business-ready data, ensuring the pipeline was reproducible and easy to reason about. Moreover, the project demonstrated practical application of core Spark concepts, including DataFrame and Spark SQL APIs, schema alignment across multiple sources, window functions, and the use of partitioned Parquet storage for efficient querying. Also, the six analytical queries produced meaningful and actionable insights, most notably the strong inverse relationship between delivery time and customer satisfaction, which identifies delivery speed as a key operational priority for the business.

Several directions exist for extending this work. First, the cleaned and integrated tables could be migrated from plain Parquet to Delta Lake, which would add ACID transaction guarantees, schema enforcement, and time-travel capabilities (Databricks.com, 2024). Second, the pipeline (currently executed manually as a sequence of notebooks) could be orchestrated as a scheduled Databricks Workflow, enabling automated and incremental processing as new data (orders) arrives. Third, automated data quality validation could be introduced, for example using a framework such as Great Expectations (Sobotík, 2021), to formally test assumptions about completeness and validity at each stage rather than relying on manual inspection. Finally, the integrated fact table provides a strong foundation for predictive analytics: the data could support a machine learning model for tasks such as customer satisfaction prediction or delivery delay forecasting, extending the project from descriptive analytics toward actionable prediction.

References

anita (2024). OLIST-MEET OLIST, THE LEADING ECOSYSTEM IN E-COMMERCE SOLUTIONS FOR SMES. [online] Innovations of the World. Available at: https://innovationsoftheworld.com/olist-meet-olist-the-leading-ecosystem-in-e-commerce-solutions-for-smes/.

Databricks.com. (2024). What is Delta Lake? | Databricks Documentation. [online] Available at: https://docs.databricks.com/aws/en/delta/.

Databricks. (2022). What is Medallion Architecture? [online] Available at: https://www.databricks.com/blog/what-is-medallion-architecture.

Holanda, P. (2024). CSV Files: Dethroning Parquet as the Ultimate Storage File Format — or Not? [online] DuckDB. Available at: https://duckdb.org/2024/12/05/csv-files-dethroning-parquet-or-not [Accessed 12 May 2026].

Kaggle (2021). Brazilian E-Commerce Public Dataset by Olist. [online] www.kaggle.com. Available at: https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce.

Meta. (2024). Olist, the new e-commerce unicorn. [online] Available at: https://metait.ai/cases/olist/.

Olist. (2026). O que é Olist? Conheça as soluções e aumente as vendas do seu negócio. [online] Available at: https://olist.com/o-que-e-olist/ [Accessed 25 May 2026].

Shaik Sameer (2025). Spark SQL Engine: How Queries Are Optimized and Executed. [online] Medium. Available at: https://medium.com/@sksami1997/spark-sql-engine-how-queries-are-optimized-and-executed-62cf5b3d3f84 [Accessed 12 May 2026].

Shafranovich, Y. (2005). Common Format and MIME Type for Comma-Separated Values (CSV) Files. [online] IETF. Available at: https://datatracker.ietf.org/doc/html/rfc4180.

Stripe.com. (2024). Boleto: an in-depth guide. [online] Available at: https://stripe.com/en-de/resources/more/boleto-an-in-depth-guide [Accessed 24 May 2026].

Tech, I. (2022). Olist. [online] Remote In Tech. Available at: https://remoteintech.company/companies/olist/.

Tomáš Sobotík (2021). How to ensure data quality with Great Expectations - Snowflake Builders Blog: Data Engineers, App Developers, AI/ML, & Data Science - Medium. [online] Medium. Available at: https://medium.com/snowflake/how-to-ensure-data-quality-with-great-expectations-271e3ca8b4b9.

Worldpopulationreview.com. (2024). Population of Cities in Brazil 2024. [online] Available at: https://worldpopulationreview.com/cities/brazil.